



Titre du document

Documentation



Alexandre FAUBLADIER-ANETTE

Institution des Chartreux

Date du document

Table des matières

Avant-propos	4
A qui s'adresse ce document	4
Comment rechercher une information dans le document	4
Glossaire	4
1. Les bases du réseau	4
1.1 Qu'est-ce qu'un réseau ?	4
1.2 Globalité et bases	4
1.2.1 Client serveur	4
1.2.2 DHCP	4
1.2.3 DNS	7
1.2.4 HTTP et HTTPS	10
Mise en perspective : que se passe-t-il quand vous ouvrez un site ?	15
1.3 Classification des réseaux	15
1.3.1 PAN, LAN, MAN, WAN	15
1.3.2 Réseau d'entreprise vs domestique vs internet	15
1.3.3 DMZ	15
1.3 Les Topologies	15
1.3.1 Topologie physique vs logique	15
1.3.2 Modèle actuelle	15
1.4 Notions de transmission	16
1.4.1 Signal analogique / numérique, bande passante, débit, latence, gigue, perte	16
1.4.2 Bits, octets, et le piège Mb/s vs Mo/s	16
1.4.3 Simplex, half-duplex, full-duplex	16
1.4.4 Unicast, multicast, broadcast, anycast	16
1.4.5 Domaine de collision vs domaine de diffusion	16
1.5 Les modèles en couches	16
1.5.1 Pourquoi découper en couches ?	16
1.5.2 Le modèle OSI	16
1.5.3 Le modèle TCP/IP	16
1.5.4 Encapsulation / désencapsulation	16
1.6 VLAN	16
1.6.1 Pourquoi utiliser les VLAN ?	16
1.6.2 Accès, Trunk, Hybride	16
1.6.3 Les normes 802.1Q	16
Gestion logiques des VLAN et utilisations pratique	16
Routage inter-VLAN	16
Les protocoles et leurs fonctionnements	16
1.7 Redondances de niveau 2	16
1.7.1 Tempête de broadcast	16
1.7.2 STP/RESTP/MSTP	16
1.7.3 Agrégations de liens	16
1.8 IPV4	17
1.8.1 Format et binaire	17
1.8.2 Masque de sous-réseau avec notation CIDR	17
1.8.3 Adresses réseau, diffusion et plage utilisable	17
1.8.4 Les classes	17
1.8.5 Les adresses privées	17
1.9 Subnetting (Découpages)	17
1.9.1 Méthodes	17
1.9.2 VLSM et FLSM	17
1.9.3 Les Super-réseaux	17

1.10	IPv6	17
1.10.1	Pourquoi (épuiement IPv4)	17
1.10.2	Notation, abréviation, préfixes	17
1.10.3	Types d'adresses : GUA, ULA, link-local, multicast	17
1.10.4	SLAAC, DHCPv6, NDP (le remplaçant d'ARP)	17
1.10.5	Cohabitation IPv4/IPv6 (double pile, tunnels)	17
1.11	Routage	17
1.11.1	Rôle du routeur, table de routage, passerelle par défaut	17
1.11.2	Lecture d'une table de routage, règle du préfixe le plus long	17
1.11.3	Routage statique vs dynamique	17
1.11.4	Protocoles à vecteur de distance vs à état de liens	17
1.11.5	RIP, OSPF, EIGRP, BGP (ce que fait chacun, quand on le rencontre)	17
1.11.6	Métriques, distance administrative, convergence	17
1.11.7	Redondance de passerelle : VRRP / HSRP	17
1.12	Translation NAT et PAT	18
1.12.1	Qu'est-ce que le NAT ?	18
1.12.2	NAT statique	18
1.12.3	Redirection de port	18
1.12.4	Limites	18
1.13	TCP	18
1.13.1	En-tête TCP et flag	18
1.13.2	Handshake	18
1.13.3	Fiabilité du protocole	18
1.13.4	Contrôle de flux	18
1.13.5	Les états de la connexion	18
1.14	UDP	18
1.14.1	Quand et pourquoi ?	18
1.14.2	TCP vs UDP	18
1.15	Messagerie et connexion à distances	18
1.15.1	Messagerie — SMTP, IMAP, POP3, SPF / DKIM / DMARC	18
1.15.2	Autres services — SSH, Telnet (et pourquoi ne pas l'utiliser), FTP/SFTP/FTPS, SMB, NFS, NTP, SNMP, Syslog, LDAP / Kerberos, RDP	18
1.16	Les équipements	18
1.16.1	Quels sont leurs utilités	18
1.16.2	Architecture d'un réseau d'entreprise	18
1.17	Le sans-fil (Wifi)	18
1.17.1	Les fréquences radios	18
1.17.2	Les normes	18
1.17.3	Les concepts principaux	18
1.17.4	La sécurité du wifi	18
1.17.5	les bonnes pratiques	19
1.18	VPN	19
1.18.1	Fonctionnement du VPN	19
1.18.2	Le site-à-site	19
1.18.3	IPsec	19
1.18.4	SSL/TLS	19
1.19	Virtualisation	19
1.19.1	vSwitch	19
1.19.2	Conteneur (Docker, Podman)	19
1.19.3	Cloud	19
1.19.4	SDN	19
20.	Boîte à outils et scénarios	19
21.	Des certifications pour aller plus loin	19
22.	Autres	19

Auteur	Version	Date
Killex8569	0.1	09/07/2026

Avant-propos

A qui s'adresse ce document

Comment rechercher une information dans le document

Ce document présente l'intégralité des éléments que nous avons pu apprendre lors de notre formations mais aussi lors de nos recherches personnels. Le contenu peut varier allant du fonctionnement d'une technologie jusqu'à l'installation ou l'utilisation de cette dernière.

Veuillez noter que ce document est une compilation d'informations collectées visant à

Glossaire

1. Les bases du réseau

1.1 Qu'est-ce qu'un réseau ?

1.2 Globalité et bases

1.2.1 Client serveur

1.2.2 DHCP

Un serveur DHCP est un serveur qui est utilisé pour distribuer automatiquement des adresses IP. Il est contacté par un client qui n'a pas d'IP permettant ainsi de pouvoir lui demander pour une durée déterminée d'obtenir une IP. Ce dernier évite ainsi de devoir configurer à la main une ou plusieurs adresses IP.

Le problème à résoudre Pour communiquer sur un réseau, une machine a besoin d'un minimum de quatre informations :

- une **adresse IP** (son identité sur le réseau) ;
- un **masque de sous-réseau** (pour savoir qui est « à côté » d'elle et qui est « ailleurs ») ;
- une **passerelle par défaut** (la sortie du réseau, généralement le routeur) ;
- un ou plusieurs **serveurs DNS** (pour traduire les noms en adresses IP). Configurer ces quatre paramètres à la main sur chaque poste est envisageable pour trois machines. Ça devient ingérable pour trois cents, et impossible dès qu'on introduit des portables, des téléphones et des invités qui vont et viennent. Sans compter le risque de **conflit d'adresses** : deux machines configurées avec la même IP, et plus rien ne fonctionne correctement pour ni l'une ni l'autre.

DHCP (*Dynamic Host Configuration Protocol*) automatise entièrement cette distribution.

Le vocabulaire indispensable

Terme	Définition
Étendue (<i>scope</i>)	La plage d'adresses que le serveur a le droit de distribuer. Ex. : 192.168.10.100 → 192.168.10.200.
Pool	Les adresses encore disponibles dans l'étendue.
Bail (<i>lease</i>)	La durée pendant laquelle une adresse est prêtée à un client. Typiquement 8 heures à 8 jours.
Réservation	Une adresse toujours attribuée à la même machine, identifiée par son adresse MAC.

Terme	Définition
Exclusion	Une plage à l'intérieur de l'étendue que le serveur n'attribuera jamais (réservée aux serveurs, imprimantes, équipements réseau).
Options	Les informations distribuées en plus de l'IP : passerelle (option 3), DNS (option 6), suffixe de domaine (option 15), serveur NTP, serveur de démarrage PXE...

Le processus DORA L'échange se déroule en quatre messages. On retient l'acronyme **DORA**.

CLIENT	SERVEUR
1. DHCP DISCOVER (broadcast)	
« Y a-t-il un serveur DHCP ? J'ai besoin d'une IP »	
----->	
2. DHCP OFFER (broadcast ou unicast)	
« Oui. Je te propose 192.168.10.152, masque /24,	
passerelle .254, DNS .10, bail 8 h »	
<-----	
3. DHCP REQUEST (broadcast)	
« J'accepte cette offre. »	
----->	
4. DHCP ACK (accusé de réception)	
« C'est noté, l'adresse est à toi. »	
<-----	
Le client peut maintenant communiquer.	

- **Discover** — le client cherche un serveur.
- **Offer** — le ou les serveurs proposent une adresse.
- **Request** — le client choisit **une** offre (la première arrivée) et l'annonce publiquement. Cette étape est diffusée en broadcast pour prévenir les autres serveurs éventuels que leur offre est déclinée et qu'ils peuvent libérer l'adresse qu'ils avaient mise de côté.
- **Acknowledge** — le serveur confirme et inscrit le bail dans sa base. **Détails techniques utiles :**
- DHCP fonctionne au-dessus d'**UDP**. Le serveur écoute sur le **port 67**, le client sur le **port 68**.
- Le client s'identifie par son **adresse MAC** (et, sur les systèmes récents, éventuellement par un identifiant client).
- Un message DHCP NAK peut remplacer le ACK : le serveur refuse (par exemple parce que le client redemande une adresse qui appartient désormais à une autre étendue).
- Un message DHCP **RELEASE** permet au client de rendre son adresse proprement (`ipconfig /release`).
Le renouvellement du bail

Un bail n'est pas éternel, et le client ne l'attend pas jusqu'au bout pour agir :

- à **50 %** de la durée du bail (temporisateur **T1**), le client contacte **directement** son serveur (en unicast) pour demander une prolongation. Dans l'immense majorité des cas, le serveur accepte et le compteur repart à zéro ;
- à **87,5 %** (temporisateur **T2**), si le serveur d'origine n'a pas répondu, le client repasse en broadcast et interroge n'importe quel serveur DHCP disponible ;
- à **100 %**, si personne n'a répondu, le client abandonne son adresse et recommence un DORA complet. C'est pour cette raison qu'un serveur DHCP peut redémarrer sans que personne ne perde le réseau :

les clients ne se manifestent qu'à mi-bail.

Quand il n'y a pas de serveur DHCP : l'APIPA Si un client Windows ne trouve aucun serveur, il s'attribue lui-même une adresse dans la plage **169.254.0.0/16** (mécanisme **APIPA**, aussi appelé *link-local*). Linux et macOS font la même chose.

Piège classique. Un poste avec une adresse en **169.254.x.x n'a pas eu de réponse DHCP**. Ce n'est pas un problème de DNS, ni de pare-feu, ni de site web. C'est presque toujours : câble débranché, mauvais VLAN, port switch en erreur, ou serveur DHCP hors service. Regarder l'adresse IP est le tout premier réflexe de diagnostic.

Le relais DHCP Le message DISCOVER est un **broadcast**. Or, un routeur ne transmet **jamais** un broadcast d'un réseau à l'autre. Conséquence directe : **un serveur DHCP ne dessert nativement que le réseau auquel il est physiquement rattaché.**

Il serait absurde de déployer un serveur DHCP par VLAN. On utilise donc un **relais DHCP** (*DHCP relay*, configuré sur les routeurs Cisco par la commande `ip helper-address`) :

1. Le routeur (ou le switch de niveau 3) intercepte le broadcast DHCP arrivant sur une de ses interfaces.
2. Il le transforme en **unicast** et le transmet à l'adresse du serveur DHCP central.
3. Il indique au passage sur quel sous-réseau la demande a été émise, ce qui permet au serveur de piocher dans la bonne étendue.
4. La réponse fait le chemin inverse. C'est le mécanisme qui permet d'avoir **un seul serveur DHCP** pour trente VLAN.

IP fixe, réservation ou dynamique ?

	Quand l'utiliser	Inconvénient
Adresse dynamique	Postes de travail, portables, téléphones, invités.	L'adresse change ; inutilisable pour un service qu'on doit joindre.
Réservation DHCP	Imprimantes, caméras, points d'accès, NAS.	Nécessite de connaître la MAC ; centralisé, donc dépendant du serveur DHCP.
Adresse statique (configurée sur la machine)	Serveurs, routeurs, switches, pare-feu, le serveur DHCP lui-même.	Configuration manuelle, risque de conflit, à documenter impérativement.

La bonne pratique. Préférer la **réservation** à l'adresse statique chaque fois que c'est possible : l'adresse est fixe du point de vue de la machine, mais elle reste documentée et modifiable depuis un seul endroit. Réserver le statique aux équipements qui doivent fonctionner même si le serveur DHCP est tombé.

Sécurité DHCP a été conçu à une époque où l'on ne se méfiait pas. Il n'y a **aucune authentification** : le client croit le premier serveur qui répond.

- **Serveur DHCP pirate** (*rogue DHCP*). N'importe qui branchant une box internet personnelle sur une prise murale devient serveur DHCP. Les postes reçoivent une mauvaise passerelle et une mauvaise IP : au mieux ils perdent le réseau, au pire tout leur trafic passe par la machine de l'attaquant (*man-in-the-middle*). C'est souvent involontaire — et c'est la panne « tout le monde a perdu le réseau ce matin » la plus fréquente qui soit.
- **Épuisement d'adresses** (*DHCP starvation*). Un attaquant demande des milliers de baux avec de fausses adresses MAC jusqu'à vider le pool. Plus aucun poste légitime ne peut obtenir d'adresse. **La parade** s'appelle le **DHCP snooping**, une fonction des switchs manageables : on déclare quels ports

sont « de confiance » (ceux menant au vrai serveur DHCP ou au relais). Toute réponse DHCP arrivant d'un port non approuvé est purement et simplement jetée.

Et en IPv6 ? IPv6 propose deux mécanismes, souvent combinés :

- **SLAAC** (*StateLess Address AutoConfiguration*) : le routeur annonce le préfixe du réseau, la machine construit elle-même la fin de son adresse. Aucun serveur nécessaire.
- **DHCPv6** : un vrai serveur, sur les ports **546** (client) et **547** (serveur), utile pour distribuer les DNS et garder une trace des attributions. ##### Diagnostic

```
# Windows
ipconfig /all           # voir l'IP, le bail, le serveur DHCP qui a répondu
ipconfig /release       # rendre l'adresse
ipconfig /renew         # en redemander une

# Linux
ip a                   # voir les adresses
sudo dhclient -v eth0  # relancer un DORA en mode verbeux
journalctl -u NetworkManager
```

Sur le serveur, la **liste des baux** indique qui a reçu quoi, quand, et pour combien de temps. C'est une source d'information précieuse pour retrouver « la machine qui avait l'IP .152 hier à 14 h ».

Ce qu'il faut retenir

1. DHCP distribue automatiquement IP, masque, passerelle et DNS, en quatre messages : **DISCOVER, OFFER, REQUEST, ACK**.
2. L'adresse est prêtée pour une durée limitée (le **bail**), renouvelée à mi-parcours.
3. Une adresse en **169.254.x.x** signifie que le DHCP n'a pas répondu.
4. Le broadcast DHCP ne traverse pas les routeurs : il faut un **relais DHCP**.
5. DHCP n'authentifie rien : un serveur pirate branché par erreur peut casser tout un réseau.

1.2.3 DNS

Le problème à résoudre Les machines communiquent avec des adresses IP. Les êtres humains sont incapables de retenir 142.250.75.238, et encore moins 2a00:1450:4007:80f::200e. Par ailleurs, une adresse IP change : un site migre d'hébergeur, un serveur est remplacé. Coder l'adresse en dur partout serait un cauchemar.

Le **DNS** (*Domain Name System*) est le service qui traduit un **nom** en **adresse IP** — et accessoirement bien d'autres choses.

L'analogie. Le DNS est l'annuaire téléphonique d'Internet. Vous connaissez le nom de la personne, vous cherchez son numéro. Sauf que cet annuaire est trop gros pour tenir en un seul volume : il est découpé, distribué, et chaque morceau est géré par quelqu'un de différent.

Une hiérarchie, lue de droite à gauche Un nom de domaine se lit **à l'envers**, de la droite vers la gauche :

```
www . boutique . exemple . fr .
|      |      |      |      |
|      |      |      |      +-- la racine (point final, implicite)
|      |      |      +----- TLD  (domaine de premier niveau)
|      |      +----- domaine de deuxième niveau
|      +----- sous-domaine
+----- l'hôte
```


- La **racine** est le point de départ, noté par un point final (**www.exemple.fr.**). On l'omet à l'écrit, mais il existe. Elle est servie par **13 groupes de serveurs racine** (identifiés de **a.root-servers.net** à **m.root-servers.net**, chacun répliqué en centaines de machines dans le monde).
- Le **TLD** (*Top Level Domain*) : **.fr**, **.com**, **.org**, **.io**, **.gouv.fr...**
- Le **domaine** que l'on achète auprès d'un bureau d'enregistrement (*registrar*).
- Les **sous-domaines** et **hôtes** que l'on crée librement à l'intérieur. L'ensemble du nom, écrit complètement, s'appelle un **FQDN** (*Fully Qualified Domain Name*).

Chaque niveau **délègue** au niveau inférieur. La racine ne connaît pas **exemple.fr**; elle sait juste à qui demander pour tout ce qui finit par **.fr**. C'est ce qui rend le système capable de gérer des centaines de millions de noms.

Les acteurs

Acteur	Rôle
Résolveur talon (<i>stub resolver</i>)	Le petit client DNS intégré à votre système d'exploitation. Il ne sait rien faire seul : il pose la question à son résolveur récursif.
Résolveur récursif	Celui que vous configurez (via DHCP en général). C'est lui qui fait tout le travail d'enquête. Typiquement : le serveur de votre entreprise, celui de votre fournisseur d'accès, ou un service public (1.1.1.1, 8.8.8.8, 9.9.9.9).
Serveurs racine	Indiquent quel serveur gère chaque TLD.
Serveurs de TLD	Indiquent quel serveur fait autorité sur un domaine donné.
Serveur autoritaire	Détient la vérité sur une zone. C'est lui qui possède réellement les enregistrements.

Une résolution, pas à pas Vous tapez **www.exemple.fr** dans votre navigateur. Voici ce qui se passe réellement, en supposant qu'aucun cache n'existe :

1. Le navigateur regarde **son propre cache**. Rien.
2. Le système regarde **son cache** et le fichier **hosts** (C:\Windows\System32\drivers\etc\hosts ou /etc/hosts). Rien.
3. Le résolveur talon envoie la question au **résolveur récursif** configuré. À partir d'ici, tout se passe côté serveur.
4. Le récursif interroge un **serveur racine** : « qui gère **.fr**? » → « voici les serveurs de l'AFNIC. »
5. Le récursif interroge un **serveur .fr** : « qui fait autorité sur **exemple.fr**? » → « voici **ns1.hebergeur.net** et **ns2.hebergeur.net**. »
6. Le récursif interroge **ns1.hebergeur.net** : « quelle est l'adresse de **www.exemple.fr**? » → « **93.184.216.34**, valable 3600 secondes. »
7. Le récursif **met la réponse en cache** et la renvoie à votre poste.
8. Votre navigateur ouvre enfin une connexion vers **93.184.216.34**. Les étapes 4 à 6 sont dites **itératives** : chaque serveur répond « je ne sais pas, mais demande à lui ». Seule l'étape 3 est **récursive** : votre poste pose une question et attend une réponse finale.

Dans la vraie vie, les caches font que les étapes 4 et 5 sont sautées la quasi-totalité du temps.

Le cache et le TTL Chaque enregistrement DNS est accompagné d'un **TTL** (*Time To Live*), exprimé en secondes. Il indique combien de temps la réponse peut être conservée en cache avant d'être considérée comme périmée.

- **TTL long** (24 h) : moins de trafic DNS, meilleures performances, mais une modification met un jour à se diffuser.
- **TTL court** (300 s) : les changements sont visibles rapidement, mais on interroge davantage les serveurs autoritaires. > **En pratique.** Avant une migration de serveur, on abaisse le TTL à 300 secondes **plusieurs jours à l'avance**, on effectue la bascule, puis on le remonte. C'est la seule façon de contrôler ce qu'on appelle abusivement la « propagation DNS » — il n'y a en réalité aucune propagation, seulement des caches qui expirent.

Les enregistrements les plus courants

Type	Rôle	Exemple
A	Nom → adresse IPv4	<code>www.exemple.fr</code> → <code>93.184.216.34</code>
AAAA	Nom → adresse IPv6	<code>www.exemple.fr</code> → <code>2606:2800:220:1::</code>
CNAME	Alias : « ce nom est un autre nom »	<code>boutique.exemple.fr</code> → <code>www.exemple.fr</code>
MX	Serveur de messagerie du domaine, avec une priorité	<code>exemple.fr</code> → 10 <code>mail.exemple.fr</code>
NS	Serveurs faisant autorité sur la zone	<code>exemple.fr</code> → <code>ns1.hebergeur.net</code>
TXT	Texte libre. Sert massivement à SPF, DKIM, DMARC et aux validations de propriété	<code>v=spf1</code> <code>include:_spf.google.com ~all</code>
PTR	Résolution inverse : adresse IP → nom	<code>34.216.184.93.in-addr.arpa</code> → <code>www.exemple.fr</code>
SOA	Paramètres de la zone (serveur maître, numéro de série, durées)	un seul par zone
SRV	Localise un service (port + hôte). Très utilisé par Active Directory et SIP	<code>_sip._tcp.exemple.fr</code>

Deux règles à connaître : un **CNAME** ne peut pas coexister avec un autre enregistrement pour le même nom, et un **MX** ne doit jamais pointer vers un **CNAME**.

Détails techniques

- Le DNS utilise le **port 53**, en **UDP** pour les requêtes courantes (rapide, une question, une réponse) et en **TCP** quand la réponse dépasse la taille d'un paquet ou pour les **transferts de zone** entre serveurs maître et esclave.
- Une **zone** est la portion de l'arborescence dont un serveur a la responsabilité.
- La **zone directe** fait nom → IP. La **zone inverse** (`in-addr.arpa` / `ip6.arpa`) fait IP → nom. Elle est indispensable pour les serveurs de messagerie, qui refusent volontiers les mails venant d'une IP sans reverse. ##### Le DNS en entreprise
- Un **DNS interne** résout les noms des machines du réseau local (`srv-fichiers.corp.exemple.local`), invisibles depuis Internet.
- Le **suffixe de recherche** (distribué par DHCP, option 15) permet de taper `srv-fichiers` au lieu du FQDN complet : le système complète tout seul.
- Le **split-horizon** (ou *split-brain*) consiste à répondre différemment selon l'origine de la question : `www.exemple.fr` renvoie l'IP privée du serveur si l'on est au bureau, et son IP publique si l'on est à l'extérieur.
- Sous **Active Directory**, le DNS n'est pas un confort : c'est un **prérequis absolu**. Les contrôleurs de domaine sont localisés par des enregistrements SRV. Un DNS mal configuré, et l'annuaire entier cesse de fonctionner. ##### Sécurité

- **Empoisonnement de cache** (*cache poisoning*) : injecter une fausse réponse dans un résolveur pour rediriger les utilisateurs vers un serveur malveillant.
- **DNSSEC** signe cryptographiquement les réponses. Il ne chiffre rien : il garantit seulement que la réponse n'a pas été falsifiée.
- **DoH** (*DNS over HTTPS*, port 443) et **DoT** (*DNS over TLS*, port 853) **chiffrent** les requêtes. Excellent pour la vie privée du grand public, mais problématique en entreprise : ils contournent le résolveur interne et donc le filtrage et la journalisation. Beaucoup d'organisations les bloquent explicitement.
- **Exfiltration par DNS** (*DNS tunneling*) : encoder des données volées dans des requêtes DNS. Le port 53 sortant étant presque toujours ouvert, c'est un canal discret et redoutable. ##### Diagnostic

```
nslookup www.exemple.fr                # partout
nslookup -type=MX exemple.fr

dig www.exemple.fr                     # Linux/macOS, bien plus riche
dig exemple.fr MX +short
dig @1.1.1.1 www.exemple.fr           # forcer un résolveur précis
dig www.exemple.fr +trace              # rejouer toute la chaîne depuis la racine

ipconfig /displaydns                   # Windows : voir le cache
ipconfig /flushdns                     # Windows : le vider
```

Le réflexe qui sauve. Si `ping 8.8.8.8` fonctionne mais que `ping google.fr` échoue, la couche réseau est saine : **le problème est le DNS**. Cette seule vérification élimine 80 % des hypothèses en cinq secondes. D'où l'adage bien connu des administrateurs : « *It's always DNS.* »

Ce qu'il faut retenir

1. Le DNS traduit des noms en adresses IP, via une **hiérarchie déléguée** lue de droite à gauche.
2. Le **résolveur récursif** fait l'enquête ; le **serveur autoritaire** détient la vérité.
3. Le **TTL** gouverne les caches — il n'existe pas de « propagation », seulement des expirations.
4. Enregistrements à connaître : **A**, **AAAA**, **CNAME**, **MX**, **NS**, **TXT**, **PTR**.
5. Port **53**, UDP puis TCP. **DNSSEC** authentifie, **DoH/DoT** chiffrent : ce n'est pas la même chose.
6. Ping l'IP puis le nom : c'est le test qui isole le DNS immédiatement.

1.2.4 HTTP et HTTPS

HTTP : le langage du Web **HTTP** (*HyperText Transfer Protocol*) est le protocole applicatif qui régit les échanges entre un **client** (navigateur, application mobile, script) et un **serveur** web. Il repose sur un principe d'une grande simplicité : **une requête, une réponse**.

Il fonctionne au-dessus de **TCP**, sur le **port 80** par défaut.

Deux caractéristiques fondamentales :

- **HTTP est textuel** (jusqu'à HTTP/2) : une requête est lisible à l'œil nu.
- **HTTP est sans état** (*stateless*) : le serveur ne se souvient de rien entre deux requêtes. Chaque requête est traitée comme si elle venait d'un inconnu. C'est précisément ce qui a rendu nécessaires les **cookies** et les **sessions**. ##### Anatomie d'une URL

```
https://www.exemple.fr:443/boutique/article?id=42&couleur=bleu#avis
|           |           |           |           |           |
schéma      hôte      port      chemin      paramètres      fragment
```

Le **fragment** (`#avis`) n'est **jamais envoyé au serveur** : il est traité uniquement par le navigateur.

Anatomie d'une requête et d'une réponse Requête :

```
GET /boutique/article?id=42 HTTP/1.1
Host: www.exemple.fr
User-Agent: Mozilla/5.0 (...)
Accept: text/html
Cookie: session=a1b2c3d4
```

Ligne 1 : la **méthode**, le **chemin**, la **version**. Lignes suivantes : les **en-têtes**. Puis, éventuellement, une ligne vide et un **corps** (pour un POST par exemple).

Réponse :

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8
Content-Length: 5124
Set-Cookie: session=a1b2c3d4; HttpOnly; Secure
Cache-Control: max-age=3600
```

```
<!DOCTYPE html>
<html>...
```

Ligne 1 : la version, le **code de statut**, sa description. Puis les en-têtes, une ligne vide, et le **corps** (le contenu réel).

Les méthodes

Méthode	Rôle	Sûre ?	Idempotente ?
GET	Lire une ressource	Oui	Oui
HEAD	Comme GET, mais renvoie les en-têtes seulement	Oui	Oui
POST	Créer, envoyer un formulaire	Non	Non
PUT	Remplacer une ressource entière	Non	Oui
PATCH	Modifier partiellement	Non	Non
DELETE	Supprimer	Non	Oui
OPTIONS	Demander ce que le serveur autorise (utilisé par CORS)	Oui	Oui

Sûre = ne modifie rien côté serveur. *Idempotente* = l'exécuter dix fois donne le même résultat qu'une fois.

Piège classique. Un GET ne doit **jamais** modifier de données. Les navigateurs, proxys et robots d'indexation se permettent de rejouer les GET librement. Un lien « supprimer » en GET, et le robot du moteur de recherche vide votre base de données.

Les codes de statut La première chiffre donne la famille :

Famille	Signification
1xx	Information (rare)
2xx	Succès
3xx	Redirection

Famille	Signification
4xx	Erreur du client
5xx	Erreur du serveur

Les plus fréquents :

Code	Nom	Ce que ça veut dire concrètement
200	OK	Tout va bien.
201	Created	La ressource a été créée (réponse à un POST).
204	No Content	Succès, mais rien à renvoyer.
301	Moved Permanently	Déplacé définitivement. Le navigateur mémorise.
302	Found	Redirection temporaire.
304	Not Modified	« Ton cache est encore bon, garde-le. »
400	Bad Request	La requête est malformée.
401	Unauthorized	Il faut s'authentifier (mal nommé : c'est « non authentifié »).
403	Forbidden	Authentifié, mais pas le droit.
404	Not Found	La ressource n'existe pas.
429	Too Many Requests	Vous allez trop vite.
500	Internal Server Error	Le serveur a planté.
502	Bad Gateway	Un intermédiaire n'a pas obtenu de réponse valide du serveur derrière lui.
503	Service Unavailable	Le serveur est surchargé ou en maintenance.
504	Gateway Timeout	Un intermédiaire a attendu trop longtemps.

Le réflexe. 4xx : c'est vous. 5xx : c'est eux. Cette distinction oriente immédiatement le dépannage : un 404 se corrige dans l'URL, un 502 se corrige sur le serveur.

Pourquoi HTTPS En HTTP simple, **tout circule en clair**. N'importe qui sur le chemin — un autre utilisateur du Wi-Fi du café, l'administrateur du réseau, un opérateur — peut lire vos identifiants, vos cookies de session, le contenu des pages, et même **modifier** le contenu à la volée (injection de publicités, de code malveillant).

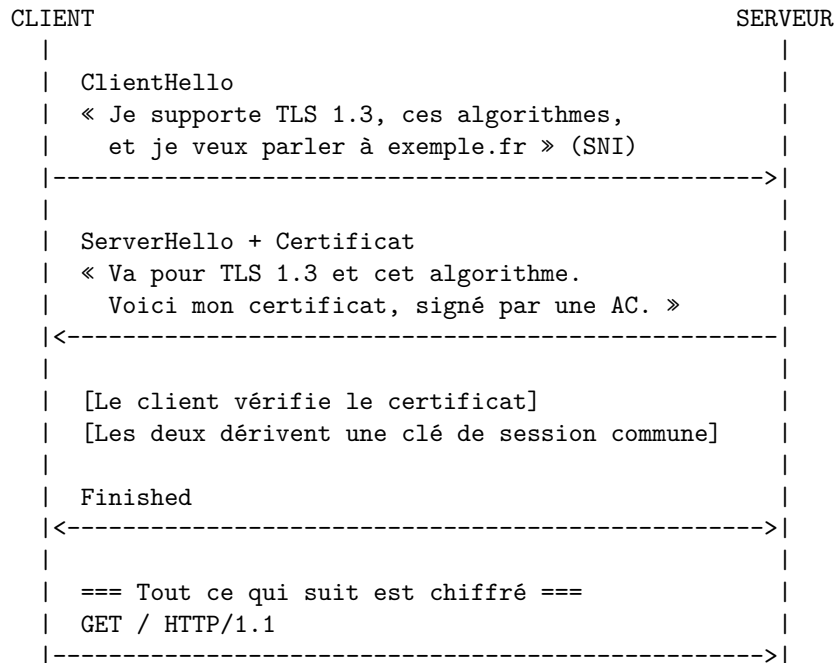
HTTPS n'est pas un autre protocole. C'est exactement **HTTP, transporté dans un tunnel TLS**. Port **443**.

HTTP	HTTPS
+-----+	+-----+
HTTP	HTTP
+-----+	+-----+
TCP	TLS <-- la seule différence
+-----+	+-----+
IP	TCP
+-----+	+-----+
	IP
	+-----+

TLS (*Transport Layer Security*, successeur de SSL) apporte trois garanties, et il faut bien les distinguer :

1. **Confidentialité** — le contenu est chiffré, personne ne peut le lire.
2. **Intégrité** — personne ne peut modifier le contenu sans que ce soit détecté.
3. **Authentification** — vous avez la preuve que vous parlez bien au serveur **exemple.fr**, et pas à un imposteur. > **La confusion à éviter.** Le cadenas ne signifie **pas** « ce site est honnête ». Il signifie « la communication avec ce site est chiffrée et je parle bien au propriétaire de ce nom de domaine ». Un site d'hameçonnage peut parfaitement obtenir un certificat gratuit et afficher un cadenas. HTTPS protège le **transport**, pas le **contenu**.

La poignée de main TLS, simplifiée



Avec **TLS 1.3**, cette négociation ne coûte qu'un **aller-retour** (contre deux en TLS 1.2). Les versions SSL 2/3 et TLS 1.0/1.1 sont **obsolètes et vulnérables** : elles doivent être désactivées.

Le certificat et la chaîne de confiance Un **certificat** associe un **nom de domaine** à une **clé publique**, et il est **signé** par une **autorité de certification** (AC).

Votre système d'exploitation et votre navigateur embarquent une liste d'**AC racines** en lesquelles ils ont confiance. La vérification consiste à remonter la chaîne :

```
Certificat de www.exemple.fr
    signé par -> AC intermédiaire
                signé par -> AC racine (présente dans le magasin du système)
```

Le navigateur vérifie aussi que le **nom du certificat correspond à l'URL**, que la **date de validité** n'est pas dépassée, et que le certificat n'a pas été **révoqué**.

Les erreurs de certificat les plus courantes :

Message	Cause réelle
<i>Certificat expiré</i>	Personne n'a renouvelé. La cause n°1 des pannes HTTPS.

Message	Cause réelle
<i>Nom ne correspond pas</i>	On accède au site par une IP, ou par un nom absent du certificat.
<i>Émetteur inconnu</i>	Certificat auto-signé, ou AC interne non installée sur le poste.
<i>Date système erronée</i>	L'horloge du poste est fausse : tous les certificats semblent invalides. Vérifiez le NTP.

Ce que HTTPS ne cache pas Point souvent mal compris, à expliquer clairement à vos collègues. Un observateur du réseau ne voit pas le contenu, mais il voit :

- l'**adresse IP** du serveur que vous contactez ;
- le **nom de domaine**, transmis en clair dans le champ **SNI** de la poignée de main (sauf si ECH est utilisé, encore rare) ;
- la **requête DNS** préalable, si elle n'est pas chiffrée ;
- le **volume** et le **rythme** des données échangées. Autrement dit : HTTPS cache **ce que vous lisez**, pas **où vous allez**.

Les versions du protocole

Version	Transport	Apport principal	Limite
HTTP/1.0	TCP	Une connexion par requête.	Très lent.
HTTP/1.1	TCP	Connexions persistantes, en-tête Host (plusieurs sites par IP).	Blocage en tête de file : une réponse lente bloque les suivantes.
HTTP/2	TCP + TLS	Binaire, multiplexage de plusieurs flux sur une connexion, compression des en-têtes.	Le blocage subsiste au niveau TCP : un paquet perdu bloque tous les flux.
HTTP/3	QUIC (UDP) + TLS 1.3	Abandonne TCP. Plus de blocage entre flux, connexion établie plus vite, survit au changement de réseau (Wi-Fi → 4G).	Parfois filtré par des pare-feu qui bloquent l'UDP sortant.

En pratique, HTTP/2 et HTTP/3 ne sont déployés qu'**en HTTPS**. Le chiffrement n'est plus une option.

Diagnostic

```
curl -I https://www.exemple.fr      # ne récupérer que les en-têtes et le code
curl -v https://www.exemple.fr     # voir tout l'échange, y compris TLS
curl -L https://exemple.fr         # suivre les redirections
```

```
openssl s_client -connect exemple.fr:443 -servername exemple.fr
# affiche le certificat, la chaîne, la version TLS négociée
```

Et, dans le navigateur, l'onglet **Réseau** des outils de développement (F12) : c'est de très loin l'outil le plus efficace pour comprendre ce qui se passe réellement lors du chargement d'une page.

Ce qu'il faut retenir

1. HTTP fonctionne en **requête / réponse**, sur TCP, port **80**. Il est **sans état**.
 2. Une requête = méthode + chemin + en-têtes (+ corps). Une réponse = **code de statut** + en-têtes + corps.
 3. **4xx : erreur du client. 5xx : erreur du serveur.**
 4. HTTPS = HTTP dans un tunnel TLS, port **443**. Il apporte confidentialité, intégrité et **authentification du serveur**.
 5. Le cadenas prouve que la communication est protégée, **pas que le site est honnête**.
 6. HTTPS masque le contenu, pas la destination (IP, SNI, DNS).
-

Mise en perspective : que se passe-t-il quand vous ouvrez un site ?

Les trois services de cette section interviennent, dans cet ordre exact. C'est le meilleur exercice de synthèse qui soit :

1. **Vous allumez votre poste.** Il n'a aucune adresse IP. Il diffuse un DHCP DISCOVER. → § 1.2.2
2. Le serveur DHCP lui attribue une IP, un masque, une **passerelle** et l'adresse d'un **serveur DNS**.
3. **Vous tapez `www.exemple.fr`.** Votre poste n'a aucune idée de l'adresse IP correspondante. Il interroge le serveur DNS reçu à l'étape 2. → § 1.2.3
4. Le DNS répond `93.184.216.34`.
5. Votre poste ouvre une **connexion TCP** vers cette adresse, sur le port **443**.
6. Une **poignée de main TLS** se déroule, le certificat est vérifié, le tunnel est chiffré. → § 1.2.4
7. Votre navigateur envoie enfin GET / HTTP/1.1. Le serveur répond 200 OK et la page s'affiche. Chacune de ces étapes peut échouer — et chacune produit un symptôme différent. Savoir **où** vous êtes dans cette séquence, c'est déjà avoir résolu la moitié du problème —

1.3 Classification des réseaux

1.3.1 PAN, LAN, MAN, WAN

1.3.2 Réseau d'entreprise vs domestique vs internet

1.3.3 DMZ

1.3 Les Topologies

bus, étoiles, maillé, hybride etc..

1.3.1 Topologie physique vs logique

1.3.2 Modèle actuelle

1.4 Notions de transmission

1.4.1 Signal analogique / numérique, bande passante, débit, latence, gigue, perte

1.4.2 Bits, octets, et le piège Mb/s vs Mo/s

1.4.3 Simplex, half-duplex, full-duplex

1.4.4 Unicast, multicast, broadcast, anycast

1.4.5 Domaine de collision vs domaine de diffusion

1.5 Les modèles en couches

1.5.1 Pourquoi découper en couches ?

1.5.2 Le modèle OSI

1.5.3 Le modèle TCP/IP

1.5.4 Encapsulation / désencapsulation

Segment → paquet → trame → bits En-têtes : qui ajoute quoi MTU, fragmentation, MSS, jumbo frames
Exemple complet : que se passe-t-il quand j'ouvre un site web (à revoir en fin de doc)

1.6 VLAN

1.6.1 Pourquoi utiliser les VLAN ?

1.6.2 Accès, Trunk, Hybride

1.6.3 Les normes 802.1Q

Gestion logiques des VLAN et utilisations pratique

Routage inter-VLAN

Les protocoles et leurs fonctionnements

1.7 Redondances de niveau 2

1.7.1 Tempête de broadcast

1.7.2 STP/RESTP/MSTP

1.7.3 Agrégations de liens

1.8 IPV4

1.8.1 Format et binaire

1.8.2 Masque de sous-réseau avec notation CIDR

1.8.3 Adresses réseau, diffusion et plage utilisable

1.8.4 Les classes

1.8.5 Les adresses privées

APIPA, loopback, réservés etc..

1.9 Subnetting (Découpages)

1.9.1 Méthodes

1.9.2 VLSM et FLSM

1.9.3 Les Super-réseaux

1.10 IPV6

1.10.1 Pourquoi (épuisement IPv4)

1.10.2 Notation, abréviation, préfixes

1.10.3 Types d'adresses : GUA, ULA, link-local, multicast

1.10.4 SLAAC, DHCPv6, NDP (le remplaçant d'ARP)

1.10.5 Cohabitation IPv4/IPv6 (double pile, tunnels)

1.11 Routage

1.11.1 Rôle du routeur, table de routage, passerelle par défaut

1.11.2 Lecture d'une table de routage, règle du préfixe le plus long

1.11.3 Routage statique vs dynamique

1.11.4 Protocoles à vecteur de distance vs à état de liens

1.11.5 RIP, OSPF, EIGRP, BGP (ce que fait chacun, quand on le rencontre)

1.11.6 Métriques, distance administrative, convergence

1.11.7 Redondance de passerelle : VRRP / HSRP

1.12 Translation NAT et PAT

1.12.1 Qu'est-ce que le NAT ?

1.12.2 NAT statique

1.12.3 Redirection de port

1.12.4 Limites

1.13 TCP

1.12.1 En-tête TCP et flag

1.12.2 Handcheck

1.12.3 Fiabilité du protocole

1.12.4 Controle de flux

1.12.5 Les états de la connexion

1.14 UDP

1.14.1 Quand et pourquoi ?

1.14.2 TCP vs UDP

1.15 Messagerie et connexion à distances

1.15.1 Messagerie — SMTP, IMAP, POP3, SPF / DKIM / DMARC

1.15.2 Autres services — SSH, Telnet (et pourquoi ne pas l'utiliser), FTP/SFTP/FTPS, SMB, NFS, NTP, SNMP, Syslog, LDAP / Kerberos, RDP

1.16 Les équipements

1.16.1 Quels sont leurs utilités

1.16.2 Architecture d'un réseau d'entreprise

1.17 Le sans-fil (Wifi)

1.17.1 Les fréquences radios

1.17.2 Les normes

1.17.3 Les concepts principaux

SSID BSSID, ESSID, WIFI 6 et 7

1.17.4 La sécurité du wifi

WEP, WPA ? WPA2, WPA3

1.17.5 les bonnes pratiques

1.18 VPN

1.18.1 Fonctionnement du VPN

1.18.2 Le site-à-site

1.18.3 IPsec

1.18.4 SSL/TLS

1.19 Virtualisation

1.19.1 vSwitch

1.19.2 Conteneur (Docker, Podman)

1.19.3 Cloud

1.19.4 SDN

20. Boite à outils et scénarios

50.1 ipconfig / ifconfig / ip a 50.2 ping, traceroute / tracert, pathping 50.3 nslookup, dig 50.4 netstat / ss, arp, route 50.5 telnet / nc pour tester un port 50.6 Wireshark / tcpdump : lire une capture sans paniquer 50.7 iperf, speedtest

21. Des certifications pour aller plus loin

22. Autres